Enabling Feedback-Driven Schema Linking for Text-to-SQL with Fine-Tuned LLMs

İlhami Uluğtürkkan
İhsan Doğramacı Bilkent University
Ankara, Türkiye
i.ulugturkkan@ug.bilkent.edu.tr

Hasan Alp Caferoğlu
İhsan Doğramacı Bilkent University
Ankara, Türkiye
alp.caferoglu@bilkent.edu.tr

Özgür Ulusoy
İhsan Doğramacı Bilkent University
Ankara, Türkiye
oulusoy@cs.bilkent.edu.tr

ABSTRACT

Accomplishing Text-to-SQL task with Large Language Models (LLMs), is a challenging and complex task. On various benchmarks such as BIRD[1] and SPIDER[2], researchers proposed promising solutions but the most accurate solution is still outperformed by human performance which has 92.96% accuracy according to BIRD-bench. While solutions differ from each other, schema-linking process is widely performed by solutions before generating SQL queries. This paper proposes a different approach to text-to-SQL task and schema-linking process by introducing a feedback mechanism during schema-linking process. Previously, a dataset was created to fine-tune the models that will be used to create the proposed system. In this work, 2 models ('Filtering-LM' and 'Feedback-LM') were fine-tuned with the created dataset and evaluated.

1 INTRODUCTION

Text-to-SQL task aims for a solution with an accuracy close to human performance accuracy. With wide usage and increasing complexities of modern database systems, the development of a highly accurate Text-to-SQL translator has become a requirement for the world. While different solution approaches [4–9]are proposed this problem, most common and accurate solution approach is to decompose the task to multiple layers. A dedicated layer —known as schema linking— aligns the user's question with the database schema to pinpoint the tables and columns relevant to the query.

This paper proposes a new schema-linking technique by using feedback mechanism among LLMs and aims to increase the accuracy of the retrieved database table names. Two models named Filtering-LM and Feedback-LM are fine-tuned for this purpose.

Filtering-LM is the model generates structured intermediate steps that indicates which instances should be used for the SQL query from the given database schema based on the question. The model generates a structured intermediate step output after analyzing the given database schema and question. Example intermediate steps can be seen in Figure 1.

On the other hand, Feedback-LM generates feedback based on the intermediate step passed to it. It also analyzes given database structure, question and additionally passed intermediate step. Feedback-LM only generates 3 types of feedback for redundant, missing, and incorrect tables or an output as 'Ground Truth' which terminates the whole process. Overall proposed solution's general structure can be seen in Figure 1.

The inspiration of this paper is real world SQL writing process of humans. Generally, people determine which table and columns to use in the SQL before writing it and if a mistake is made during this process, the final written query contains error most likely. But

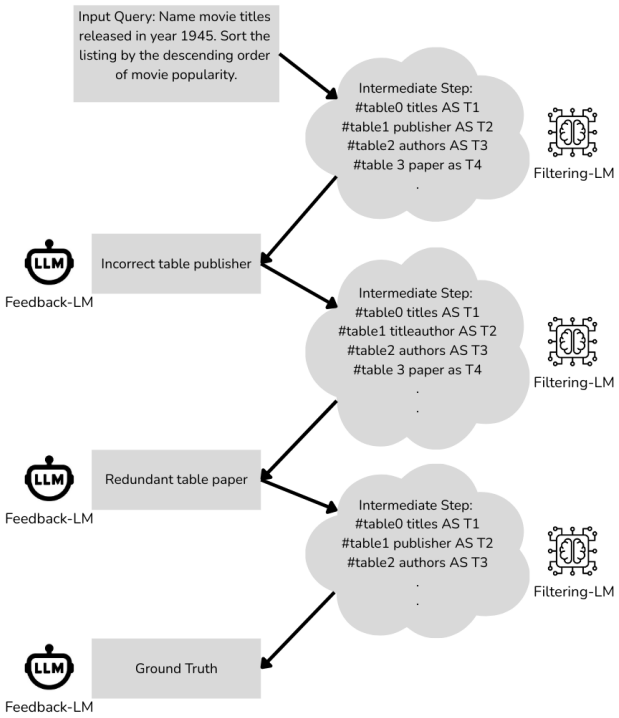


Figure 1: Targeted Structure of the Paper

if there is a second opinion on choosing which database components to choose, the process becomes more secure and more likely to be true. Past works proved that the given example is applicable for LLMs too. Explicitly generating intermediate inferences such as Chain of Thought (CoT) [3], increases the accuracy of the LLMs on reasoning tasks such as arithmetic. With CoT and other different intermediate inference generation approaches, giving beneficial feedback to LLMs and getting better result is possible [10, 11]. Therefore, it is expected to observe a better performance by decomposing the schema-linking process even further. While multiple benchmarks such as SPIDER [2] and BIRD[1] exist for evaluating text-to-SQL systems, this work focuses on the BIRD benchmark for assessment.

The rest of the paper is organized as follows. In Section 2, related works on Text-to-SQL are discussed and the necessary solution approaches are explained. Next, previously created dataset and its features are discussed in Section 3. Then, fine-tune process of the LLMs are discussed in Section 4 and evaluation of the fine-tuned

Table 1: Successes in Manipulating the Intermediate Process

	Ground Truth	Manipulated	Feedback
Deleting Table	#table0 titles AS T1 #table1 titleauthor AS T2 #table2 authors AS T3	#table0 titles AS T1 #table1 authors AS T3	missing titleauthor
Adding Table	#table0 person AS T1 #table1 taughtBy AS T2 #table2 course AS T3 #table3 advisedBy AS T4	#table0 person AS T1 #table1 taughtBy AS T2 #table2 course AS T3 #table3 advisedBy AS T4 #table4 resources AS T5	redundant resources
Changing Table	#table0 titles AS T1 #table1 titleauthor AS T2 #table2 authors AS T3	#table0 titles AS T1 #table1 publisher AS T2 #table2 authors AS T3	incorrect titleauthor

models are made in Section 5. Lastly, future works are discussed and the paper is concluded in Sections 6 and 7 respectively.

2 RELATED WORK

Related works can be categorized into two groups: Text-to-SQL Solutions and Generation of Intermediate Inferences, as the proposed solution integrates concepts and techniques from both areas.

2.1 Text-to-SQL Solutions

To propose this paper, solutions which use LLMs and decomposing techniques are examined [4–9]. Since these papers use slightly different solution techniques and models, the aim was to understand and determine the common weak sides of the solutions. After analyzing the common failures of the papers, it is concluded that the low performance of the schema-linking layer significantly contributes to the overall failure in performance. Despite having a correction level at the end of the decomposed systems, mentioned papers do not use any feedback mechanism on schema-linking layer. Since current works use fine-tuned LLMs or prompting techniques, it has been seen that this layer is open to improvement by using a feedback mechanism.

2.2 Generation of Intermediate Inferences

It is shown that a series of intermediate reasoning step improves the ability of large language models to perform complex reasoning [3]. With this inspiration, different techniques to improve LLMs performance are introduced such as feedback mechanisms. Just like Chain of Thought, the aim is to generate a intermediate step for the model to move on based on the obtained information. This way, problem can be decomposed to simple tasks and based on the simple tasks' results, a complex problem can be solved. The intermediate step generation can be done by the model itself [10], or by an intermediate step generator [11].

3 PREVIOUS WORK: DATASET CREATION

In this paper a Feedback Generator Model is proposed to improve the performance of the schema-linking layer. The general structure to be achieved can be seen in Figure 1.

As mentioned earlier, the main purpose of this work is improving the performance of the schema-linking process using intermediate inference generation method. With this structure, generated intermediate step will be criticized by the Feedback-LM which specifically fine-tuned to criticize the given intermediate step structure until there is no missing, redundant, or incorrect table names. To fine-tune such models, a specific and precisely dataset is created previously by using BIRD-bench's train set [1]. In this section, the process of creating the dataset for fine-tuning models are summarized.

3.1 Creating Ground Truth Intermediate Steps

Table 2: Structure of the train.json Entities

Entity Component	Explanation
db_id	Indicates which database to use
question	Question to generate SQL
evidence	Explanation of each SQL component
SQL	Ground Truth SQL of the question

To create the ground truth dataset, the **train.json** file of the train dataset is used. This file's context can be seen in Table 2. After deciding on the intermediate steps' format based on previous works on feedback mechanism [10, 11], SQLs of the train.json were parsed by using SQLGlot [12] and ground truth intermediate steps were generated with obtained table, column, and value names.

3.2 Creating False Intermediate Steps

The dataset to fine-tune Feedback-LM must include false intermediate steps that are highly possible to obtain with their feedback. For this purpose, intermediate steps with missing, redundant, or incorrect table names with their ground truth feedback responses are generated and stored.

3.2.1 Preparation for Intermediate Step Manipulation. To obtain the structure of all databases, the schema information from BIRD-bench's training set was used.

After extracting all the table and column names from corresponding databases, the data is stored and processed using rank_bm25

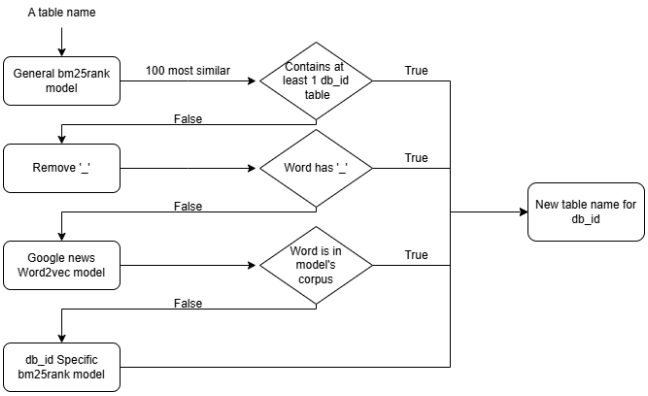


Figure 2: The Sequence of Extracting the Most Similar Table Name for a Given Table Name

models [13]. For each db_id databases a separate model is trained using its table and column names. Also, a general model had been trained with a corpus which has all the table and column names. Theses models are later used to retrieve the most similar table names for a given table name.

Besides rank_bm25 models, a TF-IDF weighted Google News Word2Vec model [14] is utilized to handle specific edge cases. This combination and other techniques which will discussed later ensure robust retrieval of relevant table names across a variety of scenarios.

3.2.2 Deleting Table Names from Intermediate Steps. To maintain generalizability, randomly chosen tables and their columns are deleted from the intermediate steps except the steps that has only 1 used table.

3.2.3 Adding Table Names to Intermediate Steps. As mentioned in earlier sections, it is important to manipulate the intermediate step in order to fine-tune the Feedback-LM for highly possible mistakes. In the literature, it has been seen that a word in the question query can be retrieved even though it is not necessary for the SQL. To replicate such cases, similar table names must be retrieved and added to the intermediate step. To achieve such scenario, the procedure showed in Figure 2 is followed.

3.2.4 Changing Table Names in Intermediate Steps. The idea is to change a used table's name with a similar table name in the intermediate step. A table name gets deleted and a similar one is inserted to the intermediate step in this process. To get a similar table name, method visualized in Figure 2 with a randomly selected table name input. Then the output is changed with the chosen table name.

The obtained data was stored on Hugging Face[1]. Some of the intermediate steps and corresponding feedback can be seen in Table 1 and training dataset feedback class distribution can be seen in Figure 3

4 FINE-TUNING LLMS

When deciding which large language model (LLM) to fine-tune for this work, Unsloth/Qwen2.5-Coder-14B-Instruct is elected over

[1]https://huggingface.co/datasets/ilhamiuturkkan/train-set

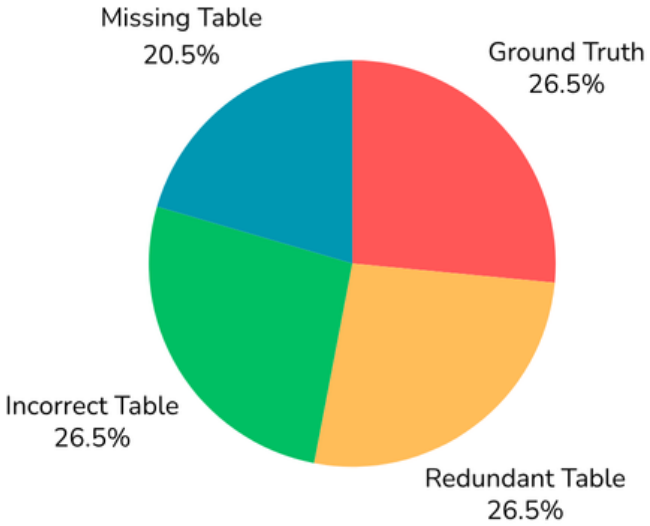


Figure 3: The Sequence of Extracting the Most Similar Table Name for a Given Table Name

other available models because this model inherently designed to handle conversational tasks effectively and its training is coder-centric. As the aim of the work is to develop a model that responds to feedback and understands the SQL language, Qwen2.5-Coder-14B-Instruct is a fine model.

Another considered feature was the computational efficiency of the model. As the working environment was Google Colab and I could not afford very long run times or high-memory footprints, the model was fine-tuned with Low-Rank Adaptation (LoRA). LoRA kept the original parameters frozen and learned only a lightweight set of rank-reduced adapter weights, so the number of trainable parameters dropped to roughly 1-2 % of the full network. Specifically, LoRA was applied a rank of 16 and lora_alpha also set to 16. The values r = 16 and lora_alpha = 16 were chosen to balance efficiency and performance, providing sufficient adaptation capacity while keeping memory and compute costs low. Consequently, the LoRA-enhanced Qwen 2.5 maintained the base model's capabilities while reducing fine-tuning time and compute costs, making the entire fine-tuning process remarkably fast and affordable on Colab.

4.1 Fine-tuning Filtering-LM

For fine-tuning the model to generate intermediate steps, we used only the ground truth examples produced based on the ground truth schemas. Additionally, the table structure of each database schema was converted into SQL Data Definition Language (DDL) statements to effectively convey the structure of the corresponding database for each question. Therefore each prompt during fine-tuning contained the objective, the database structure as DDL, the question, and a hint with the ground truth intermediate step as the wanted answer.

After setting the tokenizer truncation side to 'left', as the most valuable information accumulates at the end of the data, the hyper-parameters were configured as shown in Table 3. The number of epochs was determined based on experimental results, while the

learning rate was selected in alignment with similar works on the model and its official documentation. Other hyperparameters were chosen to ensure sustainable fine-tuning within a reasonable time.

Table 3: Critical Hyper-parameters of the Trainer for Intermediate Step Generator

Hyperparameter	Value
learning_rate	2e-4
num_train_epochs	8
weight_decay	0.01
max_seq_length	2048

4.2 Fine-tuning Feedback-LM

To fine-tune the model for feedback generation, the full dataset—including both ground truth and intentionally manipulated intermediate steps—was used. Additionally, the table structure of each database schema was converted into SQL Data Definition Language (DDL) statements to effectively convey the structure of the corresponding database for each question as done for Filtering-LM. Therefore each prompt during fine-tuning contained the objective, the database structure as DDL, the question, the hint, and a false or a correct intermediate step. As the wanted responses, ground truth feedback lines are passed to the trainer too.

As this process had much more samples to feed the model, fine-tuning the model took much more time. Therefore the model was fine-tuned for 6 epochs for nearly 46 hours on A100 GPU session. Other parameters are same as the Filtering-LM trainer.

5 EVALUATION

5.0.1 Dataset for Evaluation. For the evaluation of the models, dev-dataset of the BIRD benchmark is used. Before using it, the same procedure for the training dataset is applied. At the end of the procedure 1533 instances to evaluate Filtering-LM and 5792 instances to evaluate Feedback-LM were obtained and stored on Hugging Face[2]. Both fine-tuned models and base models were evaluated with this dataset.

5.0.2 Evaluation Process. Evaluating the responses of the LLMs presents a significant challenge in this research. While the fine-tuned models generate outputs with a fixed structure, the base models may produce more variably structured responses. To address this, different task-explanatory prompts were developed, and the resulting outputs were reviewed. After multiple iterations, the most effective prompts for both the intermediate step generator and the feedback generator were identified and are presented in Appendix A and Appendix B.

Recall emerged as the most critical evaluation metric, as the generated responses are subsequently fed into the remaining pipeline stages. Missing instances could potentially lead to severe downstream failures, making high recall a priority in this context.

While reliable results can be obtained for Filtering-LM's (both base and fine-tuned), base Feedback-LM's responses are not structured and a reliable evaluation score cannot be obtained. Even this issue addresses to the complexity of generating feedback on a intermediate step by analyzing a database schema and semantics of a question. Obtained results are given in Table 4. In Table 4, recall and precision metrics were obtained by considering the table names generated for Filtering-LM. For Feedback-LM there are 2 tasks; detecting the problematic table name and identifying the problem as missing, incorrect, redundant or ground truth.

Model	Recall	Precision
Base Intermediate Step Generator	0.5962	0.4384
Filtering-LM	0.8358	0.8621
Base Feedback Generator	ND	ND
Feedback-LM Table Selection	0.1436	0.1416
Feedback-LM Problem Identification	0.6488	0.6488

Table 4: Obtained Results of the Models

Fine-tuning had a pronounced, positive impact on the Filtering-LM. Relative to the base model, recall jumped from 0.596 to 0.836 and precision from 0.438 to 0.862, driving the F1-score from roughly 0.51 to 0.85.

By contrast, the Feedback-LM's remain bottlenecks. The fine-tuned Table-Selection variant achieves only 0.144 recall, 0.142 precision.

For problem identification part, the model achieves an overall accuracy of 0.649 across four classes. However, notably, it fails to predict the 'Ground Truth' class entirely, which raises significant concerns about insufficient representation during training. While the fine-tuning process contributed to improved performance, the absence of predictions for a full class indicates that further refinement is necessary. Overall, while the generation of intermediate reasoning steps is now robust, further gains will depend on targeted data augmentation, class-balanced or focal losses, and richer structural representations to elevate feedback generation to a comparable level of reliability.

6 FUTURE WORK

Future work will focus on improving Feedback-LM to better identify errors in intermediate reasoning steps. This includes refining its training dataset with more balanced and challenging examples, especially targeting underrepresented error types. We also plan to enhance its architecture and training procedure to boost both precision and recall, particularly in table selection and problem identification tasks. Ultimately, Feedback-LM and Filtering-LM will be integrated into a unified system, enabling real-time validation and correction of intermediate steps—strengthening the schema-linking process and improving overall text-to-SQL accuracy.

7 CONCLUSION

This study demonstrates that carefully targeted fine-tuning of a coder-centric LLM (Unsloth/Qwen 2.5-Coder-14B-Instruct) can improve the quality of intermediate reasoning steps in text-to-SQL tasks which can enhance the output of schema-linking process. By converting database schemas to DDL, creating structured

[2]https://huggingface.co/datasets/ilhamiuturkkan/dev-set-data-public

prompts, and tuning hyper-parameters within Colab constraints, intermediate-step recall is raised from 0.596 to 0.836 and precision from 0.438 to 0.862. These gains confirm that the model successfully generates intermediate steps with respect to the passed question. But also, to improve intermediate generator model's performance even more, a feedback generator is needed.

Evaluation also revealed the current bottleneck: feedback generation. Even after fine-tuning, table-selection recall hovers near 0.14 and problem-identification accuracy is 0.65, indicating that the model struggles to identify the problems on the passed intermediate steps. By leveraging a richer dataset, adopting more efficient fine-tuning techniques, and using an environment more capable than Google Colab, we can run more reliable experiments and further enhance the model's performance.

Our results show that solid intermediate steps make the whole pipeline work, but the system still needs strong feedback to succeed end-to-end. Next, we'll strengthen the feedback generator, link it more tightly to the intermediate-step module, and build a single system that catches as many SQL mistakes as possible. This way, schema-linking process of the text-to-SQL task can be improved and more accurate results can be obtained.

ACKNOWLEDGMENTS

This work was supported and advised by Prof. Dr. Özgür Ulusoy and Hasan Alp Caferoğlu.

REFERENCES

[1] J. Li, B. Hui, G. Qu, J. Yang, B. Li, B. Li, B. Wang, B. Qin, R. Geng, N. Huo, et al., "Can LLM already serve as a database interface? A big bench for large-scale database grounded text-to-SQLs," Advances in Neural Information Processing Systems, vol. 36, 2024.

[2] TaoYu,RuiZhang,KaiYang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, et al. 2018. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task.

[3] J. Wei, X. Wang, D. Schuurmans, M. Bosma, E. Chi, Q. Le, E. Choi, and D. Zhou, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," Jan. 2022. [Online]. Available: https://arxiv.org/abs/2201.11903

[4] M. Pourreza and D. Rafiei, "DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction," Advances in Neural Information Processing Systems, vol. 36, 2023. [Online]. Available: https://arxiv.org/abs/2304.11015

[5] H. Fu, C. Liu, B. Wu, F. Li, J. Tan, and J. Sun, "CatSQL: Towards Real World Natural Language to SQL Applications," Proc. VLDB Endow., vol. 16, no. 6, pp. 1534–1547, Feb. 2023. [Online]. Available: https://doi.org/10.14778/3583140.3583165

[6] S. Talaei, M. Pourreza, Y.-C. Chang, A. Mirhoseini, and A. Saberi, "CHESS: Contextual Harnessing for Efficient SQL Synthesis," 2024. [Online]. Available: https://arxiv.org/abs/2405.16755

[7] M. Pourreza and D. Rafiei, "DTS-SQL: Decomposed Text-to-SQL with Small Large Language Models," 2024. [Online]. Available: https://arxiv.org/abs/2402.01117

[8] D. Lee, C. Park, J. Kim, and H. Park, "MCS-SQL: Leveraging Multiple Prompts and Multiple-Choice Selection for Text-to-SQL Generation," 2024. [Online]. Available: https://arxiv.org/abs/2405.07467

[9] H. A. Caferoğlu and Ö. Ulusoy, "E-SQL: Direct Schema Linking via Question Enrichment in Text-to-SQL," 2024. [Online]. Available: https://arxiv.org/abs/2409.16751

[10] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, and P. Clark, "Self-Refine: Iterative Refinement with Self-Feedback," 2023. [Online]. Available: https://arxiv.org/abs/2303.17651

[11] D. Paul, M. Ismayilzada, M. Peyrard, B. Borges, A. Bosselut, R. West, and B. Faltings, "REFINER: Reasoning Feedback on Intermediate Representations," 2024. [Online]. Available: https://arxiv.org/abs/2304.01904

[12] SQLGlot. SQLGlot Documentation. Available at: https://sqlglot.com/sqlglot.html. Accessed: December 22, 2024.

[13] Dorian Brown. 2019. rank_bm25: A Collection of BM25 Algorithms in Python (Version 0.2.2). GitHub repository. Retrieved December 23, 2024, from https://github.com/dorianbrown/rank_bm25/tree/0.2.2

[14] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013. Word2Vec Tool. Google Code Archive. Retrieved December 23, 2024, from https://code.google.com/archive/p/word2vec/

A APPENDIX A: PROMPT FOR BASE FILTER MODEL

```
objective =  """Objective:
You are an intermediate step generator responsible for identifying the relevant tables needed to formulate an SQL query based on a given user question and database schema. Your task is to extract the necessary database entities by analyzing both the query and the schema. You will receive feedback based on your generated intermediate step.

Instructions:
1. Analyze the Database Schema: Review the schema to identify all tables, columns, and relationships.
2. Read the Question Carefully: Understand the focus and specific requirements of the question, including key entities and technical terms.
3. Analyze the Hint (if provided): Extract keywords or phrases that may clarify the required tables.
4. Generate the Intermediate Step: List only the table names relevant to the query in the specified format. Do not include any additional text or explanations.

**Format:**
#table0 table_name_1
#table1 table_name_2

Example:
#table0 customers
#table1 orders
#table2 order_items
"""

prompt =
    (
    f"{objective}\n"
    f"Database Schema: {e['db_schema']}\n"
    f"Question: {e['question']}\n"
    f"Hint: {e['evidence']}"
    )
```

B APPENDIX B: PROMPT FOR BASE FEEDBACK MODEL

```
objective =  """Objective:
You are a feedback generator tasked with analyzing intermediate steps in SQL query formulation. You will be provided with a database schema, a user query, and an intermediate step that outlines the relevant tables and columns necessary to construct the SQL query based on the provided schema. Your objective is to identify any errors, omissions, or redundancies in the intermediate step.

Instructions:
1. Analyze the Database Schema: Review the schema to thoroughly understand the structure, including all tables, columns, and relationships.
2. Understand the User Query: Carefully read the user query to grasp its intent, key entities, and specific requirements.
3. Analyze the Intermediate Step: Examine the provided intermediate step. Identify any irrelevant tables, missing tables, or incorrectly referenced tables.

**Important:**
- Only provide the **table names** if there is a missing, incorrect, or redundant table.
- Do **NOT** include any additional words, explanations, or sentences.
- Format the response **exactly as shown in the examples below.**

Examples:
- `incorrect elementary_schools`
- `missing ['satscores']`
- `redundant ['sat scores']`
- If the intermediate step is accurate, respond with: `Ground Truth`"""
```

```
prompt =
    (
    f"{objective}\n"
    f"Database Schema: {e['db_schema']}\n"
    f"Question: {e['question']}\n"
    f"Hint: {e['evidence']}"
    )
```